Atomcamp Cohort 18 – Computer Vision Module

Assignment: Transfer Learning & Fine-Tuning on a Custom Dataset

Context

This assignment builds directly on Days 2 & 3 of the CV module (CV Architectures & Transfer Learning). State-of-the-art computer vision models are not trained from scratch in practice but are *adapted* from powerful pretrained networks. Your goal is to experience that full pipeline on a dataset of your own choosing: from raw images to a fine-tuned model that outperforms a frozen baseline.

Learning Objectives

By completing this assignment you will be able to:

1. Load and adapt a pretrained CNN (ResNet, MobileNet, EfficientNet) for a new classification task.
2. Explain why and when to freeze vs. unfreeze layers during transfer learning.
3. Apply data augmentation strategies appropriate to the chosen model's input requirements.
4. Use learning rate scheduling to stabilize fine-tuning.
5. Critically evaluate model performance using quantitative metrics and error analysis.
6. Communicate design decisions and experimental findings clearly in a notebook.

Part 1 — Dataset Selection & Preparation

(15 marks)

1.1 Dataset Requirements

Your dataset must meet ALL of the following criteria:

- At least 5 distinct classes (categories).
- At least 200 images per class (total ≥1,000 images).
- Images must be real photographs, not illustrations, diagrams or screenshots.
- **Must NOT be CIFAR-10, MNIST, 10FoodClasses, or any dataset used in class sessions.**
- *Recommended sources: Kaggle, Roboflow, TensorFlow Datasets, Google Open Images, UCI ML Repository.*

1.2 Required Steps

- **Justify your dataset choice:** in a markdown cell, explain what the dataset is, why you chose it, and what real-world problem it relates to.
- **Folder structure:** organize images into train / val / test splits (suggested 70% / 15% / 15%). Document the split strategy.
- **Exploratory visualization:** display a 5×5 grid of sample images with class labels. Plot and discuss the class distribution and flag any imbalance.

- **Augmentation (required):** apply at least three augmentation techniques (e.g. random flip, rotation, zoom, brightness). Explain why each was chosen for your dataset. Ensure normalization matches the pretrained model's expected input (e.g. ImageNet mean/std).

Part 2 — Transfer Learning (Frozen Base)

(25 marks)

2.1 Model Setup

- **Choose one pretrained architecture:** ResNet50, MobileNetV2, EfficientNetB0, or VGG16. You must explicitly justify your choice in a markdown cell (consider parameter count, inference speed, dataset size).
- Load without the top classification layer (**include_top=False**).
- Freeze all base model layers. Verify by printing the number of trainable vs. non-trainable parameters.
- **Add a custom classification head:** at minimum GlobalAveragePooling2D → Dense (with dropout) → Softmax output. You may experiment beyond this.

2.2 Training

- Train for a minimum of 20 epochs. Use a validation set to monitor performance.
- Display the model summary (confirm base is frozen).
- Plot training and validation accuracy / loss curves. Comment on underfitting, overfitting, or convergence.
- Record your best frozen-base validation accuracy — you will compare this to fine-tuning later.

Part 3 — Fine-Tuning

(25 marks)

3.1 Unfreezing Strategy

- **Unfreeze the top layers of the base model.** Document how many layers you unfroze and why. Show a before/after trainable parameter count.
- **Use a significantly reduced learning rate** for fine-tuning (typically 10×–100× smaller than initial training LR). Explain the reasoning.
- **Apply at least one learning rate schedule:** ReduceLROnPlateau or CosineDecay are recommended. Show the schedule in action via logged LR values or a plot.

3.2 Training & Reporting

- Continue training from the frozen-base checkpoint (do not re-initialise weights).
- Plot training / validation curves for the fine-tuning phase separately.
- **Present a clear before / after comparison table:**

Metric	Frozen Base	After Fine-Tuning
Validation Accuracy		
Validation Loss		
Test Accuracy		
Trainable Params		

Part 4 — Evaluation & Analysis

(20 marks)

- **Classification report:** per-class precision, recall, and F1 score. Identify the two hardest classes to classify and hypothesize why.
- **Confusion matrix:** annotated heatmap. Point out the most common misclassification pair.
- **Error analysis:** display at least 10 misclassified images with their true and predicted labels. Look for patterns — are the errors systematic (e.g. similar classes confused) or random?
- **Performance commentary:** in a markdown cell, explain the gap between frozen-base and fine-tuned accuracy. Was the improvement what you expected?

Part 5 — Code Quality & Reflection

(15 marks)

5.1 Code Quality

- Every section must have a markdown cell at the top explaining what the following code does and why.
- Code should be modular — avoid copy-pasting the same block more than twice; use functions.
- All cells must be run in order with outputs visible before submission.
- No unnecessary debug prints or dead code left in the final submission.

5.2 Reflection Questions

Answer each of the following in a dedicated markdown cell (5–8 sentences each):

7. **Dataset choice:** Why did you pick this dataset? What practical application does it relate to? What were the main data challenges (imbalance, low quality, etc.)?
8. **Architecture choice:** Why did you choose the specific pretrained model? What trade-offs did you consider (speed vs. accuracy, model size)?
9. **Transfer learning insights:** What surprised you about the frozen-base results? Did ImageNet pretraining generalize well to your domain?
10. **Fine-tuning decisions:** How did you decide how many layers to unfreeze? What happened to training stability when you fine-tuned?
11. **What would you do next:** If you had more time and compute, what would you try to further improve performance? (e.g. more data, different architecture, Grad-CAM analysis)

Bonus Tasks (up to +20 marks)

Bonus marks are awarded for genuine depth — not quantity. Pick what interests you.

Bonus Task	Marks	Difficulty
Architecture comparison: repeat the full pipeline with a second pretrained model and compare results in a structured table.	+8	Medium
Grad-CAM visualization: implement Gradient-weighted Class Activation Mapping to show which regions of the image the model focuses on for each class.	+8	Hard
Augmentation ablation: train with no augmentation, mild augmentation, and heavy augmentation. Plot and interpret the difference in generalization.	+4	Easy

Grading Rubric

Part	Criterion	Full marks	Partial	Max
1	Dataset meets all requirements; justification is specific and thoughtful	All criteria met; clear rationale given	Minor violations or weak justification	**5**
1	EDA: sample grid, class distribution plot, augmentation with rationale	All three present and well-explained	One element missing or poorly explained	**10**
2	Model loaded correctly; base frozen; param count verified	Correct implementation with commentary	Correct but poorly explained	**10**
2	Custom head is justified; training curves shown and discussed	Head architecture explained; curves interpreted	Curves shown but no discussion	**15**
3	Unfreezing strategy explained; LR reduced; schedule applied	All three present with clear reasoning	Steps done but reasoning thin	**15**
3	Before / after comparison table complete and accurate	All metrics filled; performance gain discussed	Table present but incomplete	**10**
4	Confusion matrix + classification report with per-class analysis	Both present; hardest classes identified with hypothesis	Present but no analysis	**10**

Part	Criterion	Full marks	Partial	Max
4	Error analysis: ≥10 misclassified images; patterns identified	Examples shown; systematic patterns discussed	Examples shown but no pattern discussion	**10**
5	Code is clean, modular, well-commented; all cells run in order	Fully meets all code quality criteria	Minor issues (dead code, missing comments)	**8**
5	Reflection: all 5 questions answered thoughtfully with specific detail	Specific, insightful, goes beyond surface observations	Generic or incomplete answers	**7**
	TOTAL			**100**

Submission Instructions

Item	Details
File format	Single Jupyter Notebook (.ipynb) with all outputs visible
Filename	CV_Asg2_FirstnameLastname.ipynb
Submission via	Google Classroom

> **Academic Integrity**
> You may discuss ideas with classmates but all code and written analysis must be your own. Using AI tools (e.g. ChatGPT, Copilot) to generate large blocks of code without understanding them defeats the purpose of this assignment. If you use any AI assistance, declare it in a markdown cell and explain what you learned from it.